

Full-Stack Interview Study Notes

React & TypeScript · Python · C# & ASP.NET Core · SQL Server & MongoDB

21 lessons, condensed into concept summaries, worked examples, and the mistakes students actually make on each topic — so the explanation lands whether you get the question right or wrong.

Contents

MODULE A · REACT

- Lesson 1 — Introduction to React and JSX
- Lesson 2 — State Management with useState & useEffect
- Lesson 3 — React Router and Navigation
- Lesson 4 — Data Fetching and Context API

MODULE B · TYPESCRIPT

- Lesson 5 — TypeScript Basics and Setup
- Lesson 6 — TypeScript with React

MODULE A · REACT

- Lesson 7 — Advanced React — Custom Hooks & Performance

MODULE C · PYTHON

- Lesson 8 — Python OOP — Classes, Inheritance & Encapsulation
- Lesson 9 — Python File Handling & Data Processing
- Lesson 10 — Python Web APIs with Flask

MODULE D · C# / .NET

- Lesson 11 — C# Fundamentals & Object-Oriented Programming
- Lesson 12 — C# Control Flow & the Four Pillars of OOP
- Lesson 13 — Applying OOP — the Student Management System pattern
- Lesson 14 — Advanced C# — LINQ, Collections & Exception Handling

MODULE E · ASP.NET CORE

- Lesson 15 — Introduction to .NET & ASP.NET Core
- Lesson 16 — ASP.NET Core & Web API Development
- Lesson 17 — Entity Framework Core & Database Integration
- Lesson 18 — API Authentication & Security with JWT
- Lesson 19 — API Testing, Validation & Documentation with Swagger

MODULE F · DATABASES

- Lesson 20 — SQL Server — Queries, Joins & Database Design
- Lesson 21 — NoSQL with MongoDB & Deployment

Lesson 1: Introduction to React and JSX

React builds UIs out of reusable components instead of manually pushing and pulling the DOM by hand. JSX lets you write markup-like syntax inside JavaScript, which compiles down to plain function calls.

Key concepts

- **Virtual DOM** — React keeps an in-memory copy of the UI tree. When state changes, it diffs the new tree against the old one (reconciliation) and applies only the minimal set of real DOM updates, instead of re-rendering the whole page.
- **JSX is not HTML** — It compiles to `React.createElement(...)` calls. That's why it uses `className` instead of `class`, camelCase attributes like `onClick`, and `{}` to drop in real JavaScript expressions.
- **Functional components** — Plain JS functions that return JSX. Must be PascalCase — React treats a lowercase tag name as a built-in HTML element, not your component.
- **Props** — Read-only data passed from parent to child. Props flow one direction (down), which is what makes component composition predictable.

Example

```
function Greeting({ name }) {  
  return (  
    <div className="card">  
      <h2>Hello, {name}!</h2>  
    </div>  
  );  
}  
  
// usage  
<Greeting name="Thabo" />
```

Where students go wrong

Common mistake

Wrong: Returning two sibling elements with no wrapper: `return <h1>Hi</h1><p>Bye</p>`

Why it's wrong: JSX compiles to one function call tree with a single root node, so a component can only return one element.

Instead: wrap them in a single parent, or use a Fragment: `return <><h1>Hi</h1><p>Bye</p></>`

Lesson 2: State Management with `useState` & `useEffect`

State is what makes a component interactive. `useState` holds values that trigger a re-render when they change; `useEffect` runs code in response to render (fetching data, subscribing, syncing with the outside world).

Key concepts

- **`useState`** — Returns `[value, setValue]`. Never mutate state directly (`items.push(x)`) — always call the setter with a new value or array.
- **`useEffect` dependency array** — Controls when the effect re-runs: `[]` means 'once on mount', a list of values means 'whenever any of these change', and omitting it entirely means 'after every render'.
- **Cleanup functions** — Returning a function from `useEffect` lets you cancel timers, remove listeners, or abort fetches before the effect re-runs or the component unmounts.
- **Controlled components** — An input's value comes from state, and `onChange` updates that state — React, not the DOM, owns the source of truth.

Example

```
function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Count: ${count}`;
  }, [count]);

  return (
    <button onClick={() => setCount(c => c + 1)}>
      Clicked {count} times
    </button>
  );
}
```

Where students go wrong

Common mistake

Wrong: Writing `onClick={handleClick()}` in JSX.

Why it's wrong: The trailing `()` calls the function immediately during render, not on click — you end up running the handler once per render instead of once per click.

Instead: pass the function reference instead: `onClick={handleClick}`.

Lesson 3: React Router and Navigation

React Router turns a single-page app into something that feels like a multi-page site, swapping views based on the URL without a full page reload.

Key concepts

- **Routes** — `<Routes>` holds a list of `<Route path=... element=.../>` entries; React Router matches the current URL against them.
- **Link / NavLink** — Render an `<a>` under the hood but intercept the click to update the URL client-side. `NavLink` additionally exposes 'is this the active route' for styling.
- **Dynamic params** — A path like `/students/:id` makes `id` readable inside the routed component via `useParams()`.
- **useNavigate** — Lets you redirect programmatically — after a form submits successfully, for example — instead of requiring a user click.
- **Nested routes & catch-all** — Child routes render into a parent's `<Outlet />`; a `path="*" route` placed last catches unmatched URLs for a 404 page.

Example

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/students/:id" element={<StudentDetail />} />
  <Route path="*" element={<NotFound />} />
</Routes>

function StudentDetail() {
  const { id } = useParams();
  return <p>Showing student {id}</p>;
}
```

Where students go wrong

Common mistake

Wrong: Linking between pages with `<a href="/students">`.

Why it's wrong: A plain anchor tag forces a full browser page reload, which throws away all client-side state and defeats the point of using a single-page router.

Instead: use `<Link to="/students">` (or `NavLink`) instead.

Lesson 4: Data Fetching and Context API

Real apps pull data from a backend and often need to share state across components that aren't directly related. `useEffect` handles the first; `Context` handles the second, without threading props through every level.

Key concepts

- **Fetch inside `useEffect`** — Kick off the request after render, track loading, data, and error state separately so the UI can show the right thing at each stage.
- **Prop drilling** — Passing a value through several components that don't use it themselves, just to reach a deeply nested child. It works but becomes painful to maintain as the tree grows.
- **Context API** — `createContext` + a `Provider` makes a value available to any descendant that calls `useContext`, skipping the intermediate layers.
- **Cleanup on unmount** — If a component unmounts before a fetch resolves, calling `setState` afterwards logs a warning and can mask bugs — guard with a cleanup flag or an `AbortController`.

Example

```
const UserContext = createContext(null);

function App() {
  const [user, setUser] = useState(null);
  return (
    <UserContext.Provider value={{ user, setUser }}>
      <Dashboard />
    </UserContext.Provider>
  );
}

function Dashboard() {
  const { user } = useContext(UserContext); // no props passed down
  return <p>Welcome, {user?.name}</p>;
}
```

Where students go wrong

Common mistake

Wrong: Calling `fetch` inside `useEffect` with no dependency array, where the effect itself updates a state value that isn't in the array.

Why it's wrong: React re-runs the effect after every render that changes an untracked value it depends on, which for a fetch-then-`setState` effect creates an infinite fetch loop.

Instead: list every value the effect reads in the dependency array, and use an empty array `[]` when it should truly run once.

Lesson 5: TypeScript Basics and Setup

TypeScript adds a type layer on top of JavaScript that's checked at compile time, catching a whole class of bugs before the code ever runs.

Key concepts

- **Basic types** — string, number, boolean, arrays (`string[]`), tuples (fixed-length, fixed-type arrays), and enum for named constants.
- **interface vs type alias** — Both describe object shapes. Interfaces can be re-opened and merged, and read more naturally for public API shapes; type aliases are required for unions and intersections.
- **any vs unknown** — `any` turns off type checking entirely for that value — avoid it. `unknown` is type-safe: you must narrow it (e.g. with `typeof`) before you can use it.
- **Generics** — `<T>` lets a function or type work with whatever type is passed in, while still keeping full type checking — a reusable box instead of one function per type.

Example

```
interface User {
  id: number;
  name: string;
  role?: "admin" | "student"; // union type, optional
}

function getFirst<T>(items: T[]): T {
  return items[0];
}

const first = getFirst<User>(users); // first is typed as User
```

Where students go wrong

Common mistake

Wrong: Typing a function parameter as `any` to make an error go away.

Why it's wrong: `any` silently disables type checking for everything that value touches afterwards, so bugs it would have caught resurface at runtime instead — it's opting out of the tool you just installed.

Instead: use the real type, or `unknown` plus a narrowing check if the type is genuinely not known yet.

Lesson 6: TypeScript with React

Typing a React component means typing three things: what props it takes, what its state holds, and what shape its events are — so mistakes at the call site get caught before the browser does.

Key concepts

- **Typed props** — Define an interface for the component's props and destructure against it: `function Card({ title }: CardProps)`.
- **Typed useState** — TypeScript infers the type from the initial value, but when the initial value doesn't convey the full type — like starting at `null` — annotate it explicitly: `useState<User | null>(null)`.
- **Typed events** — React re-exports DOM event types: `React.ChangeEvent<HTMLInputElement>`, `React.MouseEvent<HTMLButtonElement>`, `React.FormEvent<HTMLFormElement>`.
- **children** — Typed as `React.ReactNode`, which covers strings, elements, fragments, and arrays of any of those.
- **React.FC** — Adds an implicit children prop and can fight with generics; many teams prefer typing the function directly instead of wrapping it in `React.FC`.

Example

```
interface ButtonProps {
  label: string;
  onClick: (e: React.MouseEvent<HTMLButtonElement>) => void;
}

function Button({ label, onClick }: ButtonProps) {
  return <button onClick={onClick}>{label}</button>;
}

const [user, setUser] = useState<User | null>(null);
```

Where students go wrong

Common mistake

Wrong: Writing `const [user, setUser] = useState(null)`; then later doing `setUser({ id: 1, name: "Amy" })`.

Why it's wrong: TypeScript infers the state's type purely from `null`, so it locks the type down to `null` and flags every later assignment as an error.

Instead: annotate the initial state explicitly: `useState<User | null>(null)`.

Lesson 7: Advanced React — Custom Hooks & Performance

As apps grow, two problems show up repeatedly: duplicated stateful logic across components, and components re-rendering or recalculating more than they need to. Custom hooks solve the first; memoization solves the second.

Key concepts

- **Custom hooks** — A function whose name starts with use that calls other hooks internally, packaging up reusable stateful logic (e.g. `useLocalStorage`).
- **React.memo** — Wraps a component so it skips re-rendering when its props are shallowly equal to last time.
- **useMemo** — Caches the *result* of an expensive calculation between renders, recomputing only when its dependencies change.
- **useCallback** — Caches a *function reference* itself, which matters when that function is passed to a memoized child or used as another hook's dependency.
- **Code splitting** — `React.lazy` + `<Suspense>` load a component's code only when it's actually needed, shrinking the initial bundle.

Example

```
function useLocalStorage(key, initial) {  
  const [value, setValue] = useState(() => {  
    const stored = localStorage.getItem(key);  
    return stored ? JSON.parse(stored) : initial;  
  });  
  
  useEffect(() => {  
    localStorage.setItem(key, JSON.stringify(value));  
  }, [key, value]);  
  
  return [value, setValue];  
}
```

Where students go wrong

Common mistake

Wrong: Wrapping every value and function in `useMemo/useCallback` 'to be safe', including a plain string concatenation.

Why it's wrong: Memoization itself has a cost (storing and comparing dependencies each render); applied to cheap values it adds overhead and complexity without any measurable benefit — it's optimizing something that was never slow.

Instead: reach for them only when the value is genuinely expensive to compute, or the reference is passed to a memoized child / used as a dependency elsewhere.

Lesson 8: Python OOP — Classes, Inheritance & Encapsulation

Object-oriented Python organizes related data and behaviour together into classes, and lets new classes reuse and specialize existing ones through inheritance.

Key concepts

- **`__init__` and `self`** — The constructor runs when an object is created; `self` is the instance being built, explicitly passed as the first parameter of every instance method.
- **Method types** — Instance methods take `self`; `@classmethod` takes `cls` and operates on the class itself; `@staticmethod` takes neither and is just a function namespaced inside the class.
- **Inheritance & `super()`** — A subclass extends a base class and can override its methods; `super().__init__(...)` calls the parent's constructor so it still runs its own setup.
- **Polymorphism / duck typing** — Python doesn't check an object's declared type before calling a method — if it has the right method, it works, regardless of its class.
- **Encapsulation convention** — A single leading underscore (`_name`) signals 'internal, please don't touch' by convention only; double underscore (`__name`) triggers name mangling, not real privacy.
- **`__str__` vs `__repr__`** — `__str__` is the readable, user-facing string; `__repr__` is the unambiguous, developer-facing one shown in a debugger or REPL.

Example

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f"{self.name} makes a sound"

class Dog(Animal):
    def speak(self): # override
        return f"{self.name} barks"

for pet in [Animal("Generic"), Dog("Rex")]:
    print(pet.speak()) # polymorphism: same call, different behaviour
```

Where students go wrong

Common mistake

Wrong: Assuming `self.__balance` inside a class makes the attribute truly private and inaccessible from outside.

Why it's wrong: Python performs name mangling on double-underscore attributes (`_ClassName__balance`), which changes the name but doesn't stop anyone from accessing it — Python has no enforced private access modifier like Java or C#.

Instead: treat the underscore convention as a documented boundary, and use `@property` if you need controlled read/write access.

Lesson 9: Python File Handling & Data Processing

Most real automation work is reading data in one shape, transforming it, and writing it back out — text files, CSVs, JSON — reliably and without leaking resources.

Key concepts

- **Context managers** — `with open(path) as f:` guarantees the file is closed when the block exits, even if an exception is raised inside it.
- **CSV & JSON** — `csv.DictReader` turns rows into dictionaries keyed by header; `json.load/json.dump` convert between JSON text and Python dicts/lists.
- **pathlib** — `Path("data") / "file.csv"` builds paths that work correctly on both Windows and Unix, instead of hand-joining strings with slashes.
- **try/except/finally** — Catch the specific exception you expect (e.g. `FileNotFoundError`) rather than a bare `except:`, so real bugs don't get silently swallowed.

Example

```
import csv
from pathlib import Path

path = Path("data") / "students.csv"

try:
    with open(path, encoding="utf-8") as f:
        reader = csv.DictReader(f)
        rows = [row for row in reader]
except FileNotFoundError:
    print(f"No file at {path}")
    rows = []
```

Where students go wrong

Common mistake

Wrong: Opening a file with `f = open(path)` and manually calling `f.close()` at the end of the function.

Why it's wrong: If an exception happens between the `open` and the `close` call, `close()` never runs and the file handle leaks — the `with`-block closes it automatically on any exit path, including an exception.

Instead: use `with open(path) as f:` instead.

Lesson 10: Python Web APIs with Flask

Flask is a lightweight framework for exposing Python logic as an HTTP API — the layer that lets a React frontend (or anything else) talk to a Python backend over the network.

Key concepts

- **Routes** — `@app.route("/students", methods=["GET", "POST"])` maps a URL and HTTP method to a Python function.
- **Reading request data** — Query params via `request.args`, a JSON body via `request.json`, form data via `request.form`.
- **jsonify + status codes** — Responses should return `jsonify(data)`, `status_code` — 200 for a successful read, 201 for a created resource, 400 for bad input, 404 for missing, 500 for a server error.
- **REST conventions** — Resource-based URLs (`/students`, `/students/<id>`) where the HTTP verb, not the URL, carries the action: GET reads, POST creates, PUT updates, DELETE removes.

Example

```
@app.route("/students", methods=["POST"])
def create_student():
    data = request.json
    if not data or "name" not in data:
        return jsonify({"error": "name is required"}), 400
    student = save_student(data)
    return jsonify(student), 201
```

Where students go wrong

Common mistake

Wrong: Returning `jsonify(student)` with no status code for every endpoint, including creation and error paths.

Why it's wrong: Flask defaults to 200 when no status is given, so a client (or Postman test, or an interviewer) checking the status code sees success even when validation actually failed.

Instead: return the tuple form and pick the status code that matches what happened: 201 for created, 400 for bad input, etc.

Lesson 11: C# Fundamentals & Object-Oriented Programming

C# is Microsoft's statically-typed, object-oriented language and the foundation for everything later in .NET — syntax, typing, and OOP building blocks that ASP.NET Core is built on top of.

Key concepts

- **Strong typing** — Every variable has a fixed, compiler-checked type: `int`, `string`, `bool`, `double`, `decimal` (for money — avoids float rounding errors), arrays.
- **Value vs reference types** — Value types (`int`, `struct`) are copied when assigned or passed; reference types (`class`) are assigned/passed by reference to the same object on the heap.
- **Access modifiers** — `public` (accessible anywhere), `private` (only inside the class), `protected` (class and subclasses) control encapsulation.
- **Naming conventions** — PascalCase for classes, methods, and public properties (`GetStudent`); camelCase for local variables and parameters (`studentId`).

Example

```
public class Student
{
    public string Name { get; set; }
    private int _age;

    public Student(string name, int age)
    {
        Name = name;
        _age = age;
    }

    public string Describe() => $"{Name}, age {_age}";
}
```

Where students go wrong

Common mistake

Wrong: Passing a `class` instance into a method, mutating a property inside it, and expecting the caller's copy to be untouched — or the reverse assumption for a `struct`.

Why it's wrong: Mixing up value and reference semantics is one of the most common C# bugs — mutating a `class` instance inside a method changes it everywhere that reference is held, while mutating a `struct` parameter only changes the local copy.

Instead: remember which kind of type you're holding: classes share one object by reference, structs are copied.

Lesson 12: C# Control Flow & the Four Pillars of OOP

Before advanced C#, everything rests on controlling program execution and structuring code around objects — encapsulation, inheritance, polymorphism, and abstraction, the four ideas every OOP interview question eventually comes back to.

Key concepts

- **Control flow** — `if/else` and `switch` for branching; `for`, `foreach`, `while`, and `do-while` (runs its body at least once, condition checked after) for looping.
- **Encapsulation** — Bundling data with the methods that operate on it, and hiding internal details behind properties/access modifiers so callers use a controlled interface.
- **Inheritance** — A subclass (`: BaseClass`) reuses and extends a base class's members.
- **Polymorphism** — The same method call behaves differently depending on the actual runtime type — in C# this requires the base method to be `virtual` (or `abstract`) and the subclass to use `override`.
- **Abstraction** — Exposing only what a consumer needs through an `abstract class` or `interface`, hiding how it's actually implemented.
- **Method overloading** — Multiple methods with the same name but different parameter lists, resolved at compile time — not the same thing as overriding, which is resolved at runtime.

Example

```
public abstract class Shape
{
    public abstract double Area();
}

public class Circle : Shape
{
    public double Radius { get; set; }
    public override double Area() => Math.PI * Radius * Radius;
}

// method overloading: same name, different signature
public int Add(int a, int b) => a + b;
public double Add(double a, double b) => a + b;
```

Where students go wrong

Common mistake

Wrong: Writing `public override string Describe()` in a subclass when the base class's `Describe()` is a plain method, not marked `virtual` or `abstract`.

Why it's wrong: This is a compile error, not a runtime surprise — and it's the tell that the student is confusing overloading (same name, different parameters, no keyword needed) with overriding (same signature, requires `virtual/override`, enables polymorphism).

Instead: mark the base method `virtual` (or make the class `abstract`) before overriding it in the subclass.

Lesson 13: Applying OOP — the Student Management System pattern

The bootcamp's recurring practical project is a console-based Student Management System: it's worth understanding as a pattern, since interviewers often ask you to design something structurally identical.

Key concepts

- **Modelling the domain** — A `Student` class holds the data (name, id, grades); a separate manager/service class holds the operations (add, search, update, list) — keeping data and orchestration logic apart.
- **Constructors for valid state** — Requiring required fields through the constructor (rather than leaving them settable after the fact) makes it harder to end up with a half-built, invalid object.
- **Collections as storage** — A console CRUD app typically holds records in a `List<Student>` in memory, standing in for a database until Entity Framework is introduced later in the module.
- **Separating concerns** — Input/output (reading console commands, printing results) stays separate from the logic that actually adds/searches/updates — the same split a real API's controller/service layers make later.

Example

```
public class StudentManager
{
    private readonly List<Student> _students = new();

    public void Add(Student s) => _students.Add(s);

    public Student? FindById(int id) =>
        _students.FirstOrDefault(s => s.Id == id);

    public IEnumerable<Student> All() => _students;
}
```

Where students go wrong

Common mistake

Wrong: Letting the console `Main` method directly manipulate the `List<Student>` everywhere input is read.

Why it's wrong: Scattering the same list-manipulation logic across every menu option makes each new feature (validation, sorting, persistence) something you have to repeat in several places instead of changing once.

Instead: route every add/search/update through the manager class's methods, and keep `Main` focused on reading input and printing output.

Lesson 14: Advanced C# — LINQ, Collections & Exception Handling

These are the tools that make everyday C# code concise instead of loop-heavy: LINQ for querying data in memory, the right collection for the access pattern you actually need, and structured error handling.

Key concepts

- **LINQ core methods** — `Where()` filters, `Select()` projects/transforms, `OrderBy()` sorts, `GroupBy()` buckets, `Join()` combines two sequences on a key.
- **Deferred execution** — A LINQ query doesn't run when you write it — it runs when you enumerate it (a `foreach`, or calling `.ToList()`). If the source changes in between, the query sees the changed data.
- **Choosing a collection** — `Dictionary<K, V>` for lookup by key, `HashSet<T>` for uniqueness checks, `Queue<T>` for FIFO, `Stack<T>` for LIFO, `List<T>` as the general-purpose default.
- **Exception handling** — `try/catch/finally`; custom exceptions inherit from `Exception` to carry domain-specific meaning (class `InvalidGradeException : Exception`).
- **Null-handling operators** — `?.` (null-conditional access), `??` (fallback if null), `??=` (assign only if currently null).

Example

```
var topStudents = students
    .Where(s => s.Average >= 70)
    .OrderByDescending(s => s.Average)
    .Select(s => s.Name)
    .ToList(); // execution actually happens here

try
{
    ValidateGrade(grade);
}
catch (InvalidGradeException ex)
{
    Console.WriteLine(ex.Message);
    throw; // preserves the original stack trace
}
```

Where students go wrong

Common mistake

Wrong: Re-throwing a caught exception with `throw ex;` instead of `throw;`.

Why it's wrong: `throw ex;` resets the stack trace to this catch block, hiding where the exception actually originated — exactly the information you need when debugging it later.

Instead: use the bare `throw;` when re-raising the exception you just caught.

Lesson 15: Introduction to .NET & ASP.NET Core

ASP.NET Core is Microsoft's modern, cross-platform framework for building web apps and APIs on top of .NET. This lesson is about the shape of a project before writing real endpoints in it.

Key concepts

- **.NET vs ASP.NET Core** — ".NET" is the runtime/platform; ASP.NET Core is the web framework built on top of it, used for both MVC (server-rendered views) and Web APIs (JSON endpoints, no views).
- **Program.cs** — The application's entry point: configures services for dependency injection, builds the middleware pipeline, and starts listening for requests — all in one file in the modern minimal-hosting model.
- **Dependency Injection** — Services are registered with a lifetime — Transient (new instance every time), Scoped (one per request), Singleton (one for the app's lifetime) — and the container hands them to whatever constructor asks for them.
- **Middleware pipeline** — Requests pass through an ordered chain (`app.Use . . .`) — logging, auth, routing, error handling — each piece able to inspect, modify, short-circuit, or pass along the request.

Example

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddScoped<IStudentService, StudentService>();

var app = builder.Build();

app.MapGet("/ping", () => "pong");

app.Run();
```

Where students go wrong

Common mistake

Wrong: Registering a Scoped service (like one holding a database context) as Singleton, or injecting a Scoped service into a Singleton.

Why it's wrong: A Singleton is built once and lives for the app's entire lifetime, so it ends up permanently holding onto whatever Scoped dependency it first received (like a database connection meant for one request), causing subtle bugs that only show up under real traffic.

Instead: match the lifetime to how the service actually needs to behave — database-backed services are almost always Scoped.

Lesson 16: ASP.NET Core & Web API Development

This is where REST principles become actual controllers: routes, HTTP verbs, and the status codes that tell a client what really happened.

Key concepts

- **Attribute routing** — `[Route("api/[controller]")]` on the class plus `[HttpGet]/[HttpPost]`/etc. on each action maps HTTP verb + URL to a method.
- **Model binding** — ASP.NET Core automatically fills a method's parameters from the route, query string, or request body — `[FromBody]` is required to bind a JSON request body to a complex parameter.
- **Returning the right status** — `Ok(data)` (200), `CreatedAtAction(...)` (201), `NotFound()` (404), `BadRequest(...)` (400) — returned as `ActionResult` or `ActionResult<T>`.

Example

```
[ApiController]
[Route("api/[controller]")]
public class StudentsController : ControllerBase
{
    [HttpGet("{id}")]
    public ActionResult<Student> GetById(int id)
    {
        var student = _service.Find(id);
        if (student is null) return NotFound();
        return Ok(student);
    }

    [HttpPost]
    public ActionResult<Student> Create([FromBody] Student s)
    {
        _service.Add(s);
        return CreatedAtAction(nameof(GetById), new { id = s.Id }, s);
    }
}
```

Where students go wrong

Common mistake

Wrong: Omitting `[FromBody]` on a POST action's complex parameter, or always returning `Ok()` even when the lookup found nothing.

Why it's wrong: Without `[FromBody]`, model binding tries the query string/route by default and the JSON body is ignored, so the parameter comes back null; always returning 200 hides real failures from any client checking the status code instead of parsing the body.

Instead: annotate body parameters with `[FromBody]`, and return `NotFound()`/`BadRequest()` for the cases that actually are those things.

Lesson 17: Entity Framework Core & Database Integration

EF Core is an ORM: it maps C# classes to database tables and lets you query/persist data as objects instead of hand-writing SQL for every operation.

Key concepts

- **DbContext & DbSet** — DbContext represents a session with the database; each DbSet<Student> property on it maps to a table.
- **Configuring a model** — Data annotations ([Key], [Required]) on the entity class, or the Fluent API inside OnModelCreating for more control.
- **Migrations** — dotnet ef migrations add InitialCreate generates a script from model changes; dotnet ef database update applies it — this is how the schema evolves alongside the code.
- **CRUD & SaveChanges** — context.Students.Add(s), .Find(id), .Remove(s) stage changes in memory; nothing hits the database until context.SaveChanges() is called.
- **Eager loading** — .Include(s => s.Courses) loads related data in the same query, avoiding a separate round-trip per row (the N+1 problem).

Example

```
public class SchoolContext : DbContext
{
    public DbSet<Student> Students => Set<Student>();
}

var student = new Student { Name = "Zanele" };
context.Students.Add(student);
context.SaveChanges(); // nothing is persisted before this line
```

Where students go wrong

Common mistake

Wrong: Calling context.Students.Add(student) and assuming the row now exists in the database.

Why it's wrong: EF Core only tracks the change in memory until SaveChanges runs — forgetting it is one of the most common reasons a 'successful' create endpoint doesn't actually persist anything.

Instead: always follow up with context.SaveChanges() (or SaveChangesAsync()) once all the changes for that unit of work are staged.

Lesson 18: API Authentication & Security with JWT

Authentication proves who's calling the API; authorization decides what they're allowed to do once identified. JWT is the standard way to carry that identity statelessly between requests.

Key concepts

- **JWT structure** — Three base64 segments — `header.payload.signature` — the payload carries claims (user id, roles, expiry), and the signature lets the server verify it hasn't been tampered with, without storing session state.
- **Login flow** — A login endpoint validates credentials, then issues a signed token; the client attaches it to later requests, typically as `Authorization: Bearer <token>`.
- **[Authorize]** — Decorating a controller or action with `[Authorize]` rejects unauthenticated requests; `[Authorize(Roles = "Admin")]` additionally requires a specific role claim.
- **Expiration & refresh** — Tokens carry a short expiry to limit the damage if one leaks; a longer-lived refresh token lets the client get a new access token without forcing a full re-login.

Example

```
[Authorize(Roles = "Admin")]
[HttpDelete("{id}")]
public IActionResult Delete(int id)
{
    _service.Remove(id);
    return NoContent();
}

// client sends:
// Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Where students go wrong

Common mistake

Wrong: Storing the JWT in `localStorage` and reading it from there on every request.

Why it's wrong: `localStorage` is readable by any JavaScript running on the page, so a single XSS vulnerability anywhere in the app can exfiltrate every logged-in user's token — an `httpOnly` cookie isn't reachable from JS at all.

Instead: prefer an `httpOnly` cookie where possible, and always set a short, enforced expiration on the token regardless of where it's stored.

Lesson 19: API Testing, Validation & Documentation with Swagger

A working API isn't a finished one: it needs to reject bad input with a clear message, and needs documentation another developer (or your future self) can actually use.

Key concepts

- **Swagger / OpenAPI** — Generates interactive, browsable documentation directly from your controllers and models — you can try a real request against the running API from the browser.
- **Model validation** — Data annotations like `[Required]` and `[StringLength(50)]` on a model; with `[ApiController]`, invalid models automatically return 400 with the validation errors before your action code even runs.
- **Client vs server validation** — Client-side checks (a required-field message in a React form) are about UX; server-side validation is about correctness and security, since any client can be bypassed by calling the API directly.
- **Testing tools** — Swagger UI for exploratory testing during development; Postman for scripted, repeatable request collections.

Example

```
public class CreateStudentRequest
{
    [Required]
    [StringLength(100)]
    public string Name { get; set; }

    [Range(0, 100)]
    public int Average { get; set; }
}

// with [ApiController], an invalid Name or out-of-range Average
// returns 400 automatically, before the action body runs
```

Where students go wrong

Common mistake

Wrong: Validating a form's required fields only in the React frontend and trusting the API doesn't need to check them again.

Why it's wrong: Anyone can skip the frontend entirely and POST straight to the API with Postman or a script, so any rule that exists only in client-side JavaScript isn't actually enforced — it's just a UX nicety.

Instead: add the matching data annotations (or explicit checks) on the API's request models, regardless of what the frontend already validates.

Lesson 20: SQL Server — Queries, Joins & Database Design

Relational databases are still the default for most business applications. This lesson is the querying and schema-design vocabulary every backend role expects you to already know.

Key concepts

- **JOIN types** — **INNER JOIN** keeps only rows that match on both sides; **LEFT JOIN** keeps every row from the left table, filling in NULLs where there's no match on the right; **FULL OUTER JOIN** keeps everything from both sides.
- **Aggregates + GROUP BY** — **COUNT/SUM/AVG/MIN/MAX** summarize a set of rows; **GROUP BY** buckets rows first, so the aggregate is computed per bucket instead of over the whole table.
- **WHERE vs HAVING** — **WHERE** filters individual rows before grouping; **HAVING** filters groups after aggregation — you can't reference an aggregate like **COUNT(*)** in a **WHERE** clause.
- **Normalization** — **1NF/2NF/3NF** progressively remove redundancy (repeating groups, partial and transitive dependencies), each level solving a specific kind of duplicated or inconsistent data.
- **Keys, constraints, indexes** — A primary key uniquely identifies a row; a foreign key enforces that a reference points to a real row elsewhere; an index speeds up reads on that column at the cost of slower writes and extra storage.

Example

```
SELECT c.Name, COUNT(e.StudentId) AS Enrolled
FROM Courses c
LEFT JOIN Enrollments e ON e.CourseId = c.Id
GROUP BY c.Name
HAVING COUNT(e.StudentId) > 5
ORDER BY Enrolled DESC;
```

Where students go wrong

Common mistake

Wrong: Writing **WHERE COUNT(*) > 5** to filter courses with more than 5 students.

Why it's wrong: **WHERE** filters rows before they're grouped, and the aggregate value doesn't exist yet at that point in query execution — SQL Server will simply reject the query. **HAVING** runs after grouping, when **COUNT(*)** is actually available.

Instead: use **HAVING COUNT(*) > 5** instead, after the **GROUP BY**.

Lesson 21: NoSQL with MongoDB & Deployment

Not every problem fits a fixed relational schema, and no application is finished until it's actually reachable on the internet. This lesson closes the module with document databases and shipping to production.

Key concepts

- **SQL vs NoSQL** — SQL databases enforce a fixed schema and are strong at relationships/joins and consistency; MongoDB stores flexible-schema JSON-like documents (BSON) in collections, trading some consistency guarantees for schema flexibility and easier horizontal scaling.
- **MongoDB basics** — A collection holds documents (roughly: a table holds rows, but each document can have different fields); CRUD is `insertOne`/`find`/`updateOne`/`deleteOne`, with filters and projections shaping what comes back.
- **Choosing between them** — Pick SQL when relationships and strict consistency matter (financial records, enrollments); pick NoSQL when the shape of the data varies a lot or needs to scale out horizontally with minimal friction.
- **Deployment split** — A React frontend is static output and deploys well to GitHub Pages; an ASP.NET Core API needs a server process and deploys to something like Azure App Service.
- **CORS** — The deployed API must explicitly allow the deployed frontend's origin, or the browser blocks the request — this is enforced by the browser, not the server, which is why it can look fine in Postman and still fail from the actual site.

Example

```
// MongoDB: find students with an average above 70, sorted
db.students.find({ average: { $gt: 70 } }).sort({ average: -1 });

// ASP.NET Core: allow the deployed frontend's origin
builder.Services.AddCors(options =>
    options.AddPolicy("Frontend", p =>
        p.WithOrigins("https://myapp.github.io").AllowAnyMethod().AllowAnyHeader()));
```

Where students go wrong

Common mistake

Wrong: Testing the deployed API from Postman, seeing it work, and shipping — then the deployed React site can't reach it.

Why it's wrong: Postman isn't a browser, so it never enforces CORS — the restriction only kicks in when a real browser makes the request from a page served on a different origin, which is exactly the situation the deployed frontend is in.

Instead: configure CORS on the API for the frontend's exact deployed origin before considering deployment done.